Preliminaries

Deep Dive into SGLang — Preliminaries (Volume I)

Yin Yang

Foundation Books Project

Where this chapter sits

This is Chapter 1 of Volume I: it hands you the vocabulary every later chapter moves around.

- A request arrives as text — but the runtime immediately holds several *views* of it: token values, scheduled rows, resident state, an output stream.
- If all of those are called "the request," the first bug is already hidden.
- Later chapters schedule, price, and coordinate these objects *independently*. First they need names.

Goal: by the end you can say which object moves, which stays resident, and which a later invariant protects.

Roadmap

The vocabulary problem

The running specimen and request

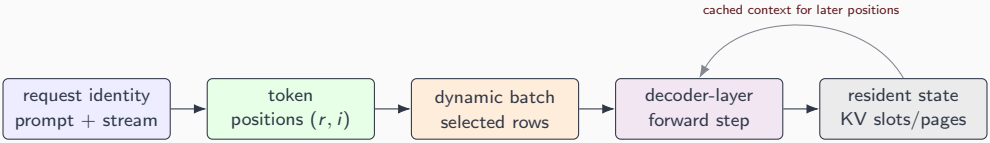
The object ledger

Concept to code

The vocabulary problem

One request, many views

The client sees one prompt and one stream. The runtime sees more:



Same request, five distinct objects. The chapter names each one and fixes what must stay true as it crosses each boundary.

The external promise hides the runtime problem

The service contract
For every accepted request, return the model's response tokens *in order* and stop at the requested stopping condition.

- One user's prompt may still be entering while another waits for its next token.
- A third request may share a reusable *prefix* with earlier traffic.
- So the runtime cannot serve one conversation at a time: it repeatedly selects pieces of work from many live requests.

That is why we need words for positions, batches, resident state, and contracts — before any mechanism appears.

The running specimen and request

Gemma 4 E4B: one fixed shape for every example

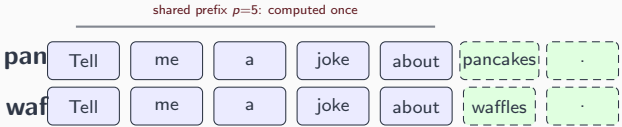
We anchor every tensor number to one text-tower specimen, `google/gemma-4-E4B`:

L layers	d_{model}	H_Q / H_{KV}	d_h
42	2560	8 / 2	256

- Used only to *fix dimensions* for runtime formulas — not as a benchmark, and no prior Gemma familiarity assumed.
- Multimodal encoders, MoE, quantization, and parallelism are deferred to Volume II.

The next chapter (The Inference Serving Problem) reuses this exact specimen.

The running request: pancakes vs. waffles



- We follow r_{pan} — a 1,000-prompt-token, 200-output-token chat request — through the whole volume.
- The pancake/waffle pair share five leading pieces: reuse is obvious to a human, correct only if positions and KV lifetime agree.

Text similarity is not reuse. Position identity is.

The object ledger

The chapter's spine: positions become rows, rows read state

1. **Positions & dynamic batches** — three requests become 34 scheduled rows, not three, not thousands.
2. **Model shape** — each row is a d_{model}-wide hidden vector through 42 decoder layers.
3. **Causal attention** — a row reads only its own earlier positions ($j \leq i$); reads grow with *resident context*.
4. **Paged KV storage** — that resident state lives in ≈ 84 KiB/token payloads, in page-aligned slots.
5. **Position validity & cost** — RoPE makes position part of the key; a small ledger counts rows, pairs, bytes.
6. **Hardware contracts** — four coarse resources and a backend contract close the loop.

The number to remember: 86,016 bytes per token

The Gemma-shaped full-history KV footprint per cached position:

$$\underbrace{42}_{L_{\text{eff}}} \cdot \underbrace{2}_{H_{\text{KV}}} \cdot \underbrace{(256{+}256)}_{d_k + d_v} \cdot \underbrace{2}_{b_{\text{elt}}} = 86{,}016 \text{ bytes} \approx 84 \text{ KiB}.$$

- Key and value dims are *added* (both stored), not multiplied.
- Cost follows $H_{\text{KV}} = 2$, not $H_Q = 8$ — that is the serving reason for grouped-query attention.

A few hundred long conversations can exhaust a GPU's KV budget. That is why memory, not FLOPs, usually bounds a serving step.

Concept to code

Every concept has a home in the source

The phase split is not just vocabulary — it is a forward mode.

from `python/sglang/srt/model_executor/forward_batch_info.py`

```python
class ForwardMode(IntEnum):
    # Extend a sequence. ... also called "prefill" in common terminology.
    EXTEND = auto()
    # Decode one token.
    DECODE = auto()
    # Contains both EXTEND and DECODE when doing chunked prefill.
    MIXED = auto()
```

Concept-to-code boxes are the spine of the book: book terms map to real classes, fields, and tensors. `EXTEND` is prefill, `DECODE` is one-row decode, `MIXED` is the running mixed step.

What to carry forward

1. **Request identity, token ID, logical position, scheduled row** answer different questions — keep them separate.
2. Reason about **scheduled rows, resident context, and KV bytes**, never request count alone.
3. **Resident is not valid:** cache reuse needs token, position, RoPE, and layout to agree.
4. **Physical $\neq$ logical:** pages, slots, and slack sit between a token length and its allocation.
5. Every object has a **home in the SGLang source**.

Next: Chapter 2 poses the serving problem in exactly these terms — which requests are live, which positions enter the next step, which resident state stays valid.